

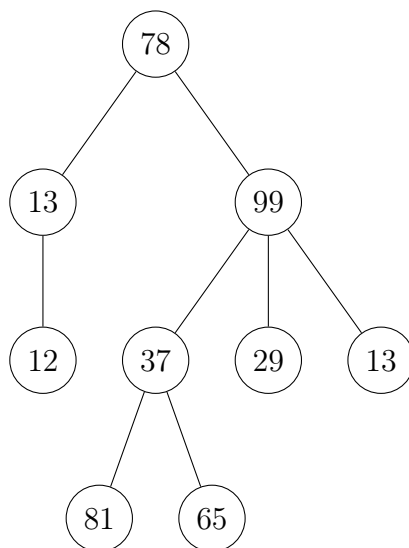
ECM306 – Tópicos Avançados em Estrutura de Dados

Simulado – Resolução

2026

Questão 1 – ENADE 2005 (Modificado)

Árvore dada:



A. Ordem de visita em pré-ordem (PREORDEM):

Pré-ordem visitamos primeiro o nó, depois recursivamente cada subárvore filha, da esquerda para a direita.

78, 13, 12, 99, 37, 81, 65, 29, 13

B. Ordem de visita em pós-ordem (POSORDEM):

Na pós-ordem visitamos recursivamente cada subárvore filha (esquerda para direita) e só depois o próprio nó.

12, 13, 81, 65, 37, 29, 13, 99, 78

Questão 2 – Profundidade do nó 99

A profundidade de um nó é o número de arestas do caminho da raiz até ele (raiz profundidade 0).

O nó 99 é filho direto da raiz (78), logo:

$$\text{Profundidade}(99) = 1$$

Questão 3 – Altura do nó 99

A altura de um nó é o comprimento do maior caminho até uma folha da sua subárvore.

Subárvore de 99:

- $99 \rightarrow 37 \rightarrow 81$ (2 arestas)
- $99 \rightarrow 37 \rightarrow 65$ (2 arestas)
- $99 \rightarrow 29$ (1 aresta)
- $99 \rightarrow 13$ (1 aresta)

O maior caminho tem 2 arestas, logo:

$$\text{Altura}(99) = 2$$

Questão 4 – ENADE 2014 (Modificado): Pilha

Comando	Estado da pilha
PUSH(p,10)	[10]
PUSH(p,5)	[10, 5]
PUSH(p,3)	[10, 5, 3]
PUSH(p,40)	[10, 5, 3, 40]
POP(p) $\rightarrow$ remove 40	[10, 5, 3]
PUSH(p,11)	[10, 5, 3, 11]
PUSH(p,4)	[10, 5, 3, 11, 4]
PUSH(p,7)	[10, 5, 3, 11, 4, 7]
POP(p) $\rightarrow$ remove 7	[10, 5, 3, 11, 4]
POP(p) $\rightarrow$ remove 4	[10, 5, 3, 11]

- Valor no topo da pilha: **11**
- Soma dos elementos: $10 + 5 + 3 + 11 = \mathbf{29}$

Questão 5 – Número máximo de nós em árvore binária de altura 10

Uma árvore binária de altura h (contando arestas, raiz com altura 0) tem no máximo:

$$N_{max} = 2^{h+1} - 1$$

Para $h = 10$:

$$N_{max} = 2^{11} - 1 = 2048 - 1 = \mathbf{2047}$$

Questão 6 – Altura de uma árvore binária completa (*full*) com 1023 nós

Para uma árvore binária cheia com $h + 1$ níveis completos:

$$N = 2^{h+1} - 1 \implies 2^{h+1} = N + 1$$

Substituindo $N = 1023$:

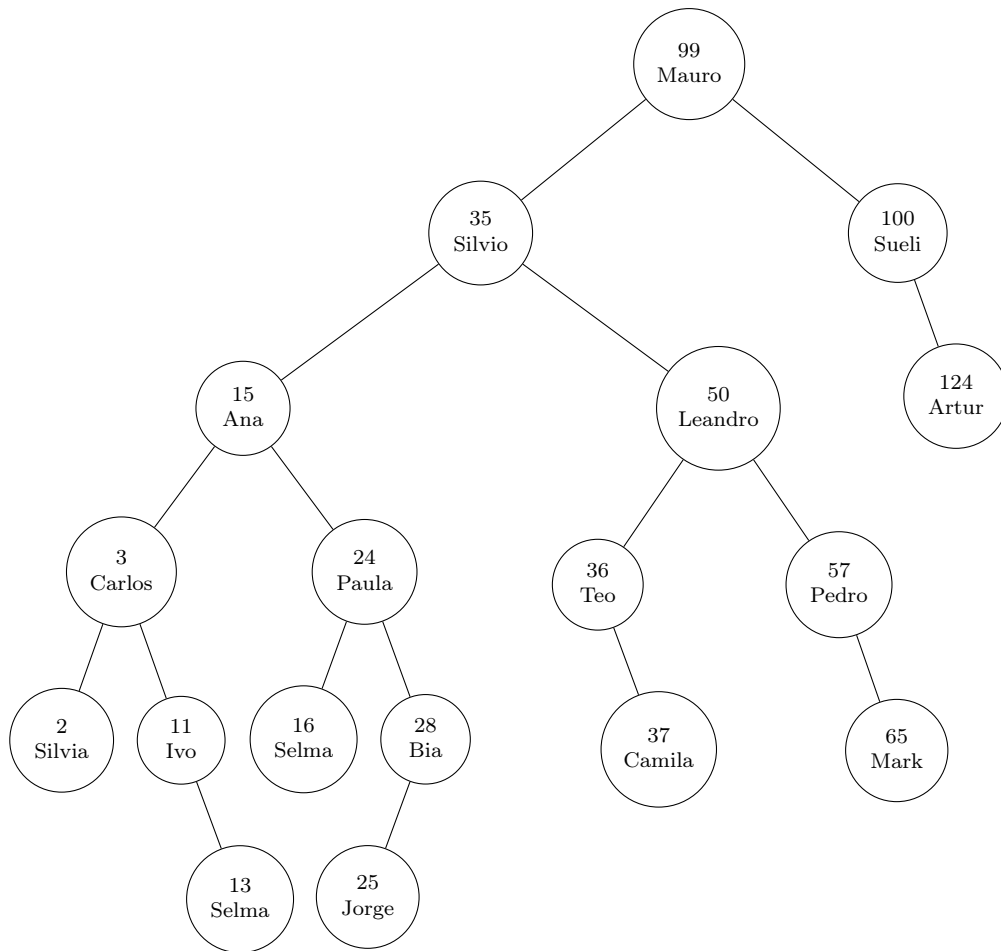
$$2^{h+1} = 1024 = 2^{10} \implies h + 1 = 10 \implies \mathbf{h = 9}$$

Questão 7 – Árvore Binária de Pesquisa (BST)

```
int [] valores =  
    {99,35,50,15,24,28,16,3,2,36,11,13,25,57,100,124,65,37};  
String [] nomes = {"Mauro","Silvio","Leandro","Ana","Paula","Bia","  
    Selma","Carlos", "Silvia","Teo","Ivo","Selma","Jorge","Pedro","  
    Sueli","Artur","Mark","Camila"};
```

A. Esboço da Arvore

Inserindo cada chave, na ordem do array, comparando com a raiz e descendo à esquerda (se menor) ou à direita (se maior), obtemos:



Trilha de inserção:

- 99 → raiz
- 35 < 99 → esquerda de 99
- 50 > 35 → direita de 35
- 15 < 35 → esquerda de 35
- 24 > 15 → direita de 15
- 28 > 24 → direita de 24
- 16 < 24 → esquerda de 24
- 3 < 15 → esquerda de 15
- 2 < 3 → esquerda de 3
- 36 < 50 → esquerda de 50
- 11 > 3 → direita de 3
- 13 > 11 → direita de 11
- 25 < 28 → esquerda de 28
- 57 > 50 → direita de 50
- 100 > 99 → direita de 99
- 124 > 100 → direita de 100
- 65 > 57 → direita de 57
- 37 > 36 → direita de 36

B. Comparações de chaves para buscar a chave 25

Percorrendo a árvore a partir da raiz:

99 → 35 → 15 → 24 → 28 → 25

Comparação	Resultado
25 vs 99	25 < 99 ⇒ vai à esquerda
25 vs 35	25 < 35 ⇒ vai à esquerda
25 vs 15	25 > 15 ⇒ vai à direita
25 vs 24	25 > 24 ⇒ vai à direita
25 vs 28	25 < 28 ⇒ vai à esquerda
25 vs 25	Encontrado!

Total: 6 comparações (99, 35, 15, 24, 28, 25).

Questão 8 – Heap: inserção de 93 e 19

Heap inicial (max-heap):

[92, 60, 78, 39, 28, 66, 70]

Inserção de 93:

Insere-se 93 na próxima posição livre (posição 8, filho da posição 4 = 39) e sobe enquanto for maior que o pai:

[92, 60, 78, 39, 28, 66, 70, 93]	93 > 39 (pai) ⇒ troca
[92, 60, 78, 93 , 28, 66, 70, 39]	93 > 60 (pai) ⇒ troca
[92, 93 , 78, 60, 28, 66, 70, 39]	93 > 92 (pai) ⇒ troca
[93 , 92, 78, 60, 28, 66, 70, 39]	93 é raiz, para

Inserção de 19:

Insere-se 19 na posição 9 (filho da posição 4 = 60):

19 < 60 ⇒ não troca, permanece no lugar

Heap resultante:

[93, 92, 78, 60, 28, 66, 70, 39, 19]

Resposta: D 93 92 78 60 28 66 70 39 19.

Código Java experimentado:

```
import java.util.PriorityQueue;
import java.util.Collections;
import java.util.ArrayList;

public class HeapTest {
    static int[] heap = new int[20];
    static int tamanho = 0;

    static void inserir(int valor) {
```

```

        tamanho++;
        heap[tamanho] = valor;
        int i = tamanho;
        while (i > 1 && heap[i] > heap[i / 2]) {
            int tmp = heap[i]; heap[i] = heap[i / 2]; heap[i / 2] =
                tmp;
            i = i / 2;
        }
    }

    public static void main(String[] args) {
        int[] inicial = {92, 60, 78, 39, 28, 66, 70};
        for (int v : inicial) inserir(v);

        inserir(93);
        inserir(19);

        System.out.print("Heap final: ");
        for (int i = 1; i <= tamanho; i++) System.out.print(heap[i]
            + " ");
        // Saída: 93 92 78 60 28 66 70 39 19
    }
}

```

Questão 9 – A sequência é um Heap?

Sequência: 33, 32, 28, 31, 26, 29, 25, 30, 27

Verificação max-heap (pai $\geq$ filhos):

Nó (índice)	Filhos (índices)	Verificação
33 (1)	32 (2), 28 (3)	$33 \geq 32$ e $33 \geq 28$ ✓
32 (2)	31 (4), 26 (5)	$32 \geq 31$ e $32 \geq 26$ ✓
28 (3)	29 (6), 25 (7)	$28 \geq 29$? FALSO

Resposta: NÃO é um Heap.

Justificativa: o nó de índice 3 (28) possui como filho esquerdo o nó de índice 6 (29). A propriedade de heap máximo (pai $\geq$ filhos) é violada nesse ponto, portanto a sequência não representa um heap válido.

Código Java experimentado (verificação):

```

public class VerificaHeap {
    static boolean isMaxHeap(int[] a, int n) {
        for (int i = 1; 2 * i <= n; i++) {
            int esq = 2 * i, dir = 2 * i + 1;
            if (esq <= n && a[i] < a[esq]) return false;
            if (dir <= n && a[i] < a[dir]) return false;
        }
        return true;
    }
}

```

```

public static void main(String[] args) {
    int[] seq = {0, 33, 32, 28, 31, 26, 29, 25, 30, 27}; // índice 0 não usado
    System.out.println("É heap? " + isMaxHeap(seq, 9)); // false
    // Falha em: a[3]=28 < a[6]=29
}
}

```

Questão 10 – Árvore com representação *Primeiro Filho* / *Próximo Irmão*

```

public class Node_Int {
    Integer item;
    Node_Int parent;
    Node_Int firstChild;
    Node_Int next;

    public Node_Int(Integer item) {
        this.item = item;
        this.parent = null;
        this.firstChild = null;
        this.next = null;
    }
}

```

A. ImprimeQtdeFilhos()

Percorre-se a lista encadeada de irmãos a partir de `firstChild`, contando os nós até encontrar `null`.

```

public class Node_Tree {

    public static void ImprimeQtdeFilhos(Node_Int no) {
        if (no.firstChild == null) {
            System.out.println("Nó sem filhos...");
            return;
        }
        int quantidade = 0;
        Node_Int filho = no.firstChild;
        while (filho != null) {
            quantidade++;
            filho = filho.next;
        }
        System.out.println("Quantidade de filhos: " + quantidade);
    }
}

```

Com base na figura, o nó A com filhos B, C, D; C com filhos E, F

```
public class TesteNodeTree {
    public static void main(String[] args) {
        Node_Int a = new Node_Int(1); // A
        Node_Int b = new Node_Int(2); // B
        Node_Int c = new Node_Int(3); // C
        Node_Int d = new Node_Int(4); // D
        Node_Int e = new Node_Int(5); // E
        Node_Int f = new Node_Int(6); // F

        a.firstChild = b;
        b.next = c;
        c.next = d;
        c.firstChild = e;
        e.next = f;

        Node_Tree.ImprimeQtdeFilhos(a); // Quantidade de filhos: 3
        Node_Tree.ImprimeQtdeFilhos(c); // Quantidade de filhos: 2
        Node_Tree.ImprimeQtdeFilhos(b); // Nó sem filhos...
    }
}
```

B. DobraValoresFilhos()

Percorre-se a mesma lista a partir de `firstChild`, multiplicando por 2 o item de cada filho direto (sem afetar netos).

```
public class Node_Tree {

    public static void DobraValoresFilhos(Node_Int no) {
        if (no.firstChild == null) {
            System.out.println("Nó sem filhos...");
            return;
        }
        Node_Int filho = no.firstChild;
        while (filho != null) {
            filho.item = filho.item * 2;
            filho = filho.next;
        }
    }
}
```

Teste:

```
public class TesteDobra {
    public static void main(String[] args) {
        Node_Int a = new Node_Int(1);
        Node_Int b = new Node_Int(10); // B
        Node_Int c = new Node_Int(20); // C
        Node_Int d = new Node_Int(30); // D
    }
}
```



```
a.firstChild = b;  
b.next = c;  
c.next = d;  
  
Node_Tree.DobraValoresFilhos(a);  
// b.item = 20, c.item = 40, d.item = 60  
  
System.out.println(b.item + " " + c.item + " " + d.item);  
// 20 40 60  
  
Node_Tree.DobraValoresFilhos(b); // Nó sem filhos...  
}  
}
```